

Reguflow - Next-Generation AI-Driven Regulatory Compliance Orchestration for Enterprise Banking

Project Overview

ReguFlow is an offline, AI-powered regulatory compliance orchestration platform designed for banking institutions. It bridges the gap between regulatory authorities and banking operations by automatically transforming newly published regulatory circulars into structured, actionable, and auditable compliance workflows.

The system operates within a fully air-gapped environment and uses locally hosted Large Language Models through Ollama, ensuring that no external APIs, cloud services, or internet connectivity are required.

ReguFlow continuously monitors the ARBI (Apex Regulatory Bank of India) Regulatory Publishing Sandbox, detects newly published circulars, extracts compliance obligations using AI, maps them into measurable action points (MAPs), routes them to the appropriate banking departments, validates implementation status, and generates tamper-evident audit trails.

The platform transforms compliance management from a manual, document-driven process into a deterministic, explainable, and traceable operational workflow.

Problem Statement

Banks receive large volumes of regulatory circulars, advisories, cyber-security mandates, fraud alerts, and operational guidelines from regulatory authorities.

Current compliance workflows face several challenges:

- Manual interpretation of lengthy regulatory documents.
- Delayed identification of actionable obligations.
- Incorrect routing of responsibilities.
- Lack of centralized compliance tracking.
- High dependency on human coordination.
- Difficulty proving compliance during audits.
- Risk of missed deadlines and regulatory penalties.

These challenges increase operational risk and slow down compliance execution.

Proposed Solution

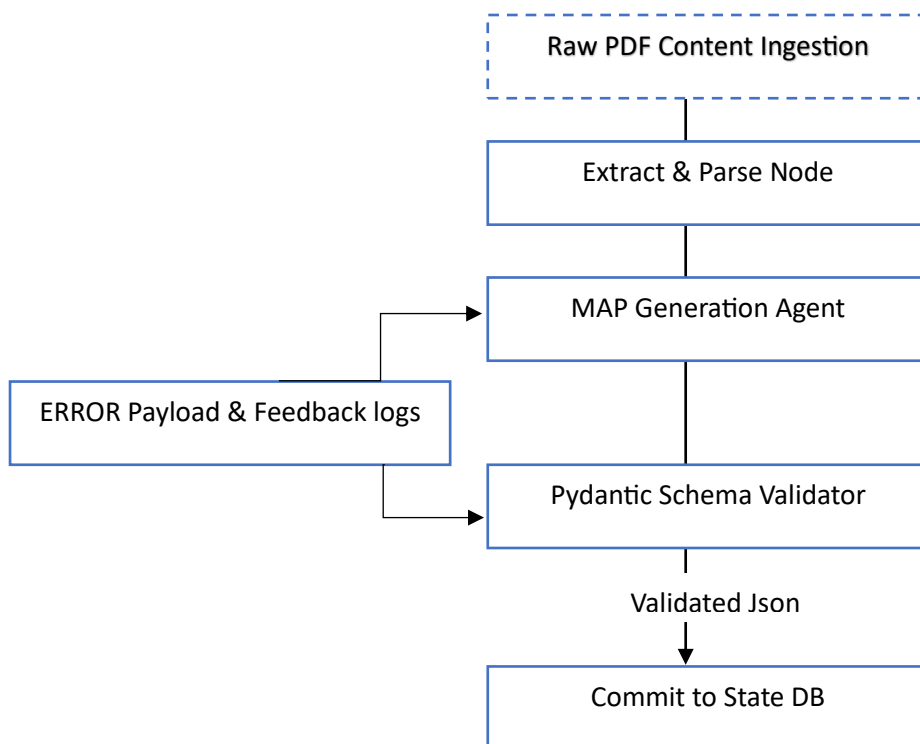
ReguFlow acts as a Regulatory Intelligence and Compliance Orchestration Layer between the regulator and the bank.

Instead of merely storing regulations, the platform:

1. Detects newly published circulars.
2. Extracts obligations using local AI models.
3. Converts obligations into structured Measurable Action Points (MAPs).
4. Maps each action to predefined banking verticals and departments.
5. Routes tasks to responsible teams.
6. Simulates compliance validation.
7. Maintains an immutable audit trail.

- Generates compliance progress reports.

System Architect:



Real Life example understanding -

here's a concrete example of how your **agentic AI layer** would work in a banking setup for an RBI-style notice about Aadhaar seeding.

Simple scenario

Assume RBI issues a notice saying:

- Every eligible citizen must link their Aadhaar to their bank account.
- Banks must complete this within a deadline.
- The bank must support government benefit transfer eligibility.
- Every action must be auditable and compliant.

Your agentic layer sits **between the regulatory input and the bank's internal systems**. It does not directly change the core banking ledger; instead, it reads the notice, converts it into tasks, routes them, checks compliance, and supervises execution. This is the safest and most realistic way to use agentic AI in banking.

What happens step by step

1. RBI notice comes in

The bank's compliance feed or regulatory team uploads the RBI circular. The agentic layer reads the notice and extracts the key instruction:

- who is affected,
- what action is required,

- deadline,
- exceptions,
- risk level,
- systems impacted.

2. LLM converts prose into structured tasks

The LLM does **not** decide policy by itself. It turns the notice into a machine-readable checklist, such as:

- identify accounts without Aadhaar mapping,
- notify customers,
- collect consent,
- verify Aadhaar details,
- update CBS,
- update NPCI mapper if needed,
- log audit evidence,
- escalate exceptions.

This is the part where your idea becomes powerful: the AI turns a vague circular into **action points** that departments can actually execute.

3. Workflow agent routes the tasks

The orchestration layer sends different tasks to different teams or systems:

- Compliance team gets the regulatory interpretation.
- Operations team gets the seeding workflow.
- Branch team gets customer support steps.
- IT team gets system-change tickets.
- Audit team gets traceability logs.

This is similar to your uploaded concept, where regulatory prose is transformed into measurable action points and routed through organizational stakeholders.

4. Customer verification happens through approved channels

The bank can expose safe channels for Aadhaar seeding, for example:

- branch counter,
- internet banking,
- mobile banking,
- ATM-based flow,
- OTP-based or consent-based request flow.

The agentic layer can guide the customer journey, but the actual verification must happen through bank-approved authentication and consent mechanisms, not by the LLM directly touching personal identity data without controls.

5. Core banking system is updated

After validation, the banking system updates:

- the CBS customer record,
- Aadhaar-link status,
- consent and verification flags,
- downstream mapper or benefit-transfer linkage where applicable.

The important point is that the agentic layer only **requests** the action through a controlled interface. The final write happens in the bank's normal transactional system, under business rules and audit controls.

6. Validation agent checks completion

A second agent or validation service verifies:

- did the account get seeded correctly,
- was the consent captured,
- is the record consistent across systems,
- did the exception queue remain empty or get properly handled.

If something fails, it opens a case for manual review instead of pretending success. This is one of the most important parts of making the system bank-safe.

7. Audit trail is created

Every step is logged:

- notice received,
- extracted instruction,
- routing decision,
- customer notification,
- verification result,
- final update,
- human approval if required,
- exception handling.

This gives the bank an evidence trail for internal audit, regulators, and future dispute resolution.

What your agent actually does

Your agentic AI should behave like a **regulatory operations copilot**, not a free-form chatbot. In this Aadhaar example, it should:

- read the notice,
- detect affected account categories,
- create action items,
- draft customer communication,
- route tasks to the right teams,
- monitor completion,

- surface exceptions,
- generate compliance status reports.

That is exactly the kind of safe enterprise use case that agentic compliance systems are being built for.

What it must not do

It should **not**:

- directly modify customer identity records without validation,
- bypass consent,
- auto-approve edge cases,
- use public-model APIs for sensitive PII,
- skip human review for risky exceptions,
- silently write to the core without logs.

The Difference Maker between the preexisting and our agentic solution.

Feature Dimension /	Pre-Existing Systems (Status Quo)	Your Agentic Architecture (The Winner)	Why Your System Wins (The Value Add)
1. Document Ingestion & Reading	Keyword & OCR Dependent Systems scan for words like "MCLR" or "KYC." If the RBI phrases a rule uniquely, the system misses it entirely.	Semantic Context Parsing LLM agents read the document like a senior lawyer, separating mandatory directives from filler text.	Zero Missed Rules: Prevents compliance leaks caused by changing regulatory language.
2. Policy Interpretation	Human-Dependent Bottleneck Legal and compliance teams must spend days reading a 100-page document to extract actionable points.	Autonomous Deconstruction The agent instantly breaks down complex prose into discrete, machine-actionable mandates.	Speed: Reduces the translation timeline from weeks to seconds .
3. Downstream Task Creation	Manual Ticket Creation Compliance officers manually type out Jira tickets, tasks, or Excel sheets to pass to the IT/Systems team.	Deterministic JSON Serialization The agent outputs a strict schema mapping the target department, deadline, risk tier, and variables.	Eliminates Human Error: No mistyping parameters or misinterpreting rules during data entry.
4. Organizational Routing	Static, Brittle Workflows Uses rigid, hardcoded email rules. If a department changes its structure, the notification chain breaks.	Dynamic Vector Graph Mapping A semantic Knowledge Graph maps the JSON payload to technical stakeholders based on the <i>meaning</i> of the update.	Zero Operational Silos: Bypasses bureaucracy to put the mandate in front of the exact engineer who needs it.
5. Integration Lifecycle	Disconnected Workflows The compliance software	Closed-Loop Automation The system functions as a	True Continuity: Compliance shifts from a

	is completely detached from the core banking software (CBS) or code base.	continuous pipeline from the RBI portal directly to the test environment.	disjointed project into an automated microservice.
6. Compliance Validation	Reactive Post-Facto Auditing Human auditors manually sample system configurations months down the line, often discovering errors too late.	Autonomous Integration Testing A validation agent automatically pings sandboxes, runs mock tests, and verifies the system configuration.	Proactive Defense: Guarantees 100% compliance <i>before</i> production deployment, avoiding heavy RBI penalties.
7. Audit Trail & Proof	Paper & Digital Logs Relies on manual email approvals, meeting minutes, and system change logs scattered across departments.	Cryptographic Verification The validation agent seals the successful system test with a tamper-proof cryptographic proof.	Unassailable Evidence: Provides the RBI with undeniable, mathematical proof of immediate compliance.
8. Architecture & Stability	Monolithic or Standard AI Either uses rigid legacy code or risky, open-ended "black box" AI chat interfaces prone to hallucinations.	Deterministic Agentic State Machine Wraps fluid LLM intelligence inside a strict, predictable framework like LangGraph or AutoGen.	Banking-Grade Trust: Combines advanced AI reasoning with 100% predictable software execution.
9. Data Privacy & Deployment	Cloud-Dependent / Third-Party Many modern AI tools require passing internal documents and system structures through external public APIs.	Air-Gapped, On-Premises Stack Optimised microservices running open-weight local models inside the bank's secure private cloud.	DPDP Act Compliant: Guarantees no sensitive banking infrastructure blueprints or PII leak outside the bank.

Simple Formula at last to check: -

1. Core Operational Environment

- **100% Offline / Air-Gapped Execution:** The system must function entirely without internet connectivity. All AI inference, orchestration, and database operations must be contained within the local hardware environment.
- **Ollama Local Inference Cluster:** The backend must interface with a local Ollama service to manage and execute multiple LLMs (Llama 3, Qwen 3, and Gemma 4) locally.
- **Resource-Efficient Architecture:** The system must be optimized to run on standard commercial laptop hardware by utilizing quantization and intelligent model switching to manage VRAM constraints.

2. Regulatory Data Processing Pipeline

- **Deterministic Event Detection:** The system shall feature a background daemon that continuously monitors the regulatory portal (Mock ARBI) for new circulars and advisories.
- **Dual-Model Ensemble Pattern:**

- **Extractor Node:** An LLM-based agent (e.g., Qwen 3) responsible for structured semantic parsing of unstructured regulatory text into JSON objects.
- **Auditor Node:** A secondary validation agent (e.g., Gemma 4) responsible for peer-reviewing extraction accuracy and flagging potential prompt-injection or hallucination.
- **Rigid Pydantic Schema Enforcement:** All extracted data must adhere to strict Pydantic models to ensure machine-readability and direct integration with downstream bank banking systems.

3. Organizational Mapping & Governance

- **Deterministic Routing Engine:** The system must utilize the provided organizational mapping matrix (theme_2.xlsx) to route extracted tasks to specific Business Verticals (e.g., *Cybersecurity Wing, IT Vertical*) and their corresponding Sub-Verticals.
- **Hybrid Governance Workflow:** The system must support two execution paths:
 - **Auto-Approval Mode:** For routine, low-risk, or previously validated compliance tasks.
 - **Human-in-the-Loop Approval:** Mandatory human review for critical directives, infrastructure modifications, or low-confidence AI outputs.

4. Data Persistence & Security

- **Document-Oriented Database:** The system shall utilize MongoDB for storage, allowing for flexible JSON-based document handling that matches the output schema of the LLM pipeline.
- **Cryptographic Audit Engine:** Every compliance action (extraction, routing, implementation, verification) must be hashed using SHA-256 to create an immutable, tamper-evident audit chain.
- **WORM Audit Integrity:** Audit logs must be structured to ensure "Write-Once-Read-Many" integrity, enabling regulators to replay the full compliance lifecycle.

5. Verification & Compliance Enforcement

- **Continuous Compliance Monitoring:** The system shall implement an automated verification engine that checks the current state of system configurations against the extracted mandates.
- **One-Click Compliance Patching:** The interface must provide an enforcement capability that allows authorized bank personnel to execute automated scripts to remediate non-compliant system states.
- **Exception Handling:** Any task failing automated verification or identified as a policy conflict must be automatically escalated for manual intervention rather than forcing an "invalid success" state.

